# DEVOPS-006 Task 3

# DEVOPS-006 — Docker Fundamentals & Containerization

## Task 3 — Run Your First Docker Container

---

#### Command run:

```
sudo docker run hello-world
```

#### What actually happens when you run this command

Breaking down the flow: **docker → run → image → container → process**

1. **docker** — you're invoking the Docker CLI.
2. **run** — you're telling Docker to create and start a container.
3. **image** — Docker first checks if the `hello-world` image exists locally.
    - It doesn't exist yet the first time, so Docker downloads ("pulls") it from **Docker Hub**.
4. **container** — Docker uses that image to create a new container (a running instance of the image).
5. **process** — Inside that container, Docker starts a single process. For `hello-world`, that process simply prints a confirmation message ("Hello from Docker!") explaining that your installation is working, and then it **exits immediately** because its job is done — it has no long-running service to keep it alive.

So in short: Docker pulled the `hello-world` image (if not already cached), created a container from it, ran a tiny process inside that container which printed a message, and the process then finished and exited.

#### Checking containers

```
sudo docker ps
```

```
sudo docker ps -a
```

#### Question: Why does `docker ps` show nothing while `docker ps -a` shows the hello-world container?

- **`docker ps`** only lists containers that are **currently running**.
- The `hello-world` container's job was just to print a message to the screen — once it finished doing that, its process **exited**, so the container immediately moved to a **"Exited"** state. Since it's no longer running, it doesn't show up in `docker ps`.
- **`docker ps -a`** shows **all containers**, regardless of state — running, stopped, or exited. Because the `hello-world` container still exists on disk (it isn't deleted automatically), it shows up here with a status like `Exited (0) X seconds ago`.

**Key takeaway:** A container isn't automatically removed once its process exits — it just stops running. It still exists (with its filesystem state) until you explicitly remove it using `docker rm <container_id>`, or run it with the `--rm` flag so it auto-deletes on exit.

---

*Deliverable: Day6 Docker Theory — Task 3 — Sprint 9 (29/6 - 12/7), DEVOPS-006*